

Response to Reviewers

PoolParty: streamlined design of DNA sequence libraries in Python

Zhihan Liu, Aidan Cordero, Justin B. Kinney

We thank the Editor and Reviewers for their thoughtful critiques, to which we provide a point-by-point response below. The manuscript has been improved substantially as a result of this feedback. Additional edits have also been made throughout the manuscript to improve clarity.

Editor Comments

In general, the reviewers had positive impressions of PoolParty, and it can likely be revised to a state that will be acceptable. Please carefully address each point raised by the reviewers. It's especially important to provide benchmarking data, comparisons to existing tools, and not to overstate the benefits the tool may provide or fail to discuss limitations. Reviewer 1 recommends comparing PoolParty to 2–3 other comparable tools. However, rather than arbitrarily picking two or three other tools for comparisons, you should use a well-defined process for picking which tools, and how many, to include in comparisons. Perhaps 2–3 is adequate; however, that may be too few tools for comparisons. The comparisons should include, at a minimum, benchmarking (e.g., runtime, memory usage), program features, and similarities/differences in program outputs.

We now provide a more thorough comparison between PoolParty and other available tools for desinging sequence libraries, including in the Introduction, Discussion and a new Supplemental Information Supplemental document. Tables S2 and S3 compare PoolParty to a wide range of available tools, and Figs. S3 and S4 compare PoolParty to its two closest competitors VaLiAnT and tangermeme, in specific use cases. We additionally attempted to provide a use case comparison to MPRAnator, but this tool runs through a web interface that is no longer functional. We have also benchmarked the compute and memory requirements of PoolParty on the three examples feature in the main text (Fig. S2 and Table S1). Resource usage is shown only for PoolParty, as none of the competing tools were able to generate the specific libraries shown in those examples.

Reviewer 1

1. In the GB1 example, the library exceeds 540,000 sequences, yet the paper provides no runtime or memory consumption benchmarks. Readers cannot assess the tool's practical usability across different library scales. The authors should supplement benchmarking data, including runtime and peak memory usage across libraries of increasing sizes (e.g., 1K, 10K, 100K, 500K sequences), and specify the test environment configuration.

The revised text benchmarks the resource requirements of PoolParty for each of the three examples featured in the main text (in Figs. 1, 2, and 3), using libraries of increasing size. The results are reported in the new Fig. S1 and Table S1. The new Supplemental Information file additionally reports the machine configuration uses in these benchmarking tests. This new material is referenced in the Discussion.

2. The paper highlights design cards as a core feature, emphasizing their use as covariates in downstream analyses. However, it does not evaluate whether design card variables provide

significant incremental predictive value beyond sequence-derived features. The authors should either supplement a comparative experiment quantifying the added information gain of design cards, or explicitly state in the Discussion that this validation is planned as future work.

Design cards record the specific design choices used to construct each sequence. We claim in the paper that this can facilitate surrogate modeling studies by making these design choices readily available as covariates, and support this claim by illustrating in Fig. 3 an sample analysis that does this. The does not, however, claim that the information in design cards increases the predictive performance of sequence-to-function models.

3. The introduction mentions VaLiAnT and MPRAnator, noting their limitations, but does not provide a systematic comparison between PoolParty and these tools. The authors should provide a table comparing PoolParty with 2–3 existing tools on core features, and demonstrate code conciseness in at least one common use case, to help readers objectively assess PoolParty’s advantages.

Tables S2 and S3 in the new Supplemental Information document provide a systematic comparison between PoolParty and other competing tools. Figs. S3 and S4 further provide code comparisons between PoolParty and the two competing packages, VaLiAnT and tangermeme, in two specific use caes. As mentioned above, we additionally tried to provide a comparison to MPRAnator, but that software (which runs through a web interface) is no longer functional. The Introduction and Discussion have also been revised to more explicitly compare PoolParty to existing tools.

4. Provide benchmarking data on runtime and memory usage across libraries of varying sizes.

This is addressed in the response to Comment 1 above.

5. Include a table comparing features with 2–3 existing tools.

This is addressed in the response to Comment 3 above.

6. To further contextualize this research within the existing academic landscape and strengthen its theoretical support, it is recommended that the authors supplement several relevant representative references.

[1] <https://doi.org/10.1093/nsr/nwaf495>

[2] <https://doi.org/10.1038/s41467-026-72882-y>

[3] <https://doi.org/10.1186/s13059-025-03606-6>

[4] <https://doi.org/10.1016/j.patcog.2025.112078>

Having read these papers, we do not see how these references are relevant to PoolParty for the more general problem of sequence library design: the first paper discusses multimodal representations of small molecules for drug discovery; the second paper models single-cell transcriptomic and surface-protein measurements; the third paper discusses read mapping to pangenome graphs;

and the fourth paper concerns circRNA-miRNA interaction prediction. We therefore decided to cite these papers in the manuscript.

7. Either provide empirical analysis of design cards’ incremental contribution to surrogate modeling performance, or explicitly discuss this as future work.

This is addressed in the response to Comment 2 above.

Reviewer 2

1. Additional statistical readouts describing pool generation would be useful. For example, how many unique sequences vs duplicates were produced in each pool (i.e. out of the universe of all permutations that meet the specifications)? How unique are the sequences (min/max/average hamming distance)? How frequent are homopolymers? Etc.

We agree that these statistics are useful for checking the generated library. We have added a `.stats()` method that can be called on any DNA Pool in the DAG, summarizing the size of the specified design space when it is fixed, the number of unique/duplicated sequences generated, pairwise Hamming distances, homopolymer frequencies, and several other basic properties of the Pool. We have also added documentation and examples at <https://poolparty.readthedocs.io/en/latest/pool.html#pool-stats>. We have added a short paragraph to the Sequence metadata subsection of the Implementation section:

PoolParty also provides a `stats` method for summarizing a DNA Pool, including its unique and duplicate sequence counts, pairwise Hamming distances, and homopolymer frequencies.

2. The authors disclose that LLMs were used to develop, debug, and document the code. Did the authors also write a test harness, and what edge cases were nominated by the LLMs for testing vs what edge cases were nominated by the human developers for testing?

Yes. PoolParty has over 3,000 tests written with pytest. These tests are from a combination of humans and coding agents. We did not track the origin of every individual test, so we cannot separate them exactly after the fact. In our experience, coding agents are particularly good at testing mechanical aspects of the code (e.g., invalid inputs and boundary conditions) instead of biologically realistic tests (e.g., DMS and MPRA designs that respect biological and physical constraints), and we added a lot of tests for real use cases during a long period of development and trial use.

3. The backward/forward pass steps are not well described in the manuscript, and figure 1b is hard to interpret. I suggest adding a supplemental figure showing how these passes are performed. For example, the text explains that the backward pass happens as information is passed to the root node – does this mean that the state is passed from ‘Mutagenize’ to ‘Pool A’ in or first from ‘Pool Final’ to ‘Stack’? And if so, what information is passed? Perhaps showing each step in an expanded figure or labeling arrows with the order would be more useful.

We agree that our description of state assignment (the backward pass) and sequence construction (the forward pass) was not clear. We have revised the Sequence generation subsection to specify the order of the two passes and what is passed between Pools and Operations. We have also expanded the example in Fig. 1B into Additional file 1: Fig. S1, numbering each step and showing the state calculations and sequence changes in action. We have revised the Sequence generation subsection as follows:

To generate a sequence library, PoolParty iterates through a series of states. Each state is an integer that, together with a random seed, uniquely determines a sequence. The generation of each individual sequence proceeds in two steps: state assignment and sequence construction (Fig. 1B; Additional file 1: Fig. S1). This process begins when the corresponding state is passed to the root Pool. State assignment involves a backward pass through the DAG, from the root Pool toward the source Pools. Each Pool passes the state it receives to the Operation that created it. Each Operation then decomposes this state into an internal state (which it keeps) and zero or more component states, each of which is passed to a distinct input Pool of that Operation. Sequence construction then occurs during a forward pass through the DAG, from the initial Pools toward the root Pool: each Operation constructs a sequence based on its internal state and the sequences returned by its input Pools, then passes this sequence to the Pool it creates. The final constructed sequence is then received from the root Pool and recorded.

4. A comparison to the pools produced by other tools such as VaLiAnT or MPRAnator would provide additional confidence in the ability of PoolParty to produce sequence pools as expected. Metrics such as the number of pool elements that overlap and an investigation of any unique pool elements would ensure that the generated pools are accurate.

We discuss this comparison in our response to Reviewer 1, Comment 3. The sequence-set comparisons are shown in Additional file 1: Figs. S3 and S4.

5. additional operations to improve library content would be useful. E.g. to maximize hamming distance, remove sequences containing restriction enzyme sites, remove homopolymers, etc. would make the tool more useful in generating ready-to-use pools.

PoolParty does have a general **filter** Operation that lets users provide a Python function to decide which sequences should be kept. It also has several convenient filter methods for basic sequence properties, including GC content, homopolymers, sequence complexity, and restriction enzyme recognition sites. We realized that these features were not clearly described in the manuscript or documentation, and have now added a short description to the manuscript and expanded the documentation. We also clarified PoolParty's limitation in the Discussion to distinguish filtering from sequence optimization. Maximizing Hamming distance and other constraint-based optimizations usually require comparing sequences across the Pool and are outside the current scope of PoolParty. We have added the following sentence to the Pools and Operations subsection:

Sequences can be filtered based on sequence length, GC content, homopolymer length, sequence complexity, restriction enzyme recognition sites, or other user-defined criteria.

We have also revised the following sentence in the limitations paragraph of the Discussion:

PoolParty is also not a sequence optimization tool; it does not design sequences that have optimized codon usage (Swainston et al., 2017), that are subject to complex synthesis constraints (Zulkower and Rosser, 2020), or that have optimal activity as assessed by machine learning models (Schreiber et al., 2025).

6. The name PoolParty may be confused with the existing PoolParty analysis software (<https://doi.org/10.1111/1755-0998.12784>).

We thank the reviewer for pointing this out. The previously published PoolParty is a pipeline for aligning and analyzing pooled-sequencing data, whereas our package is used to design DNA sequence libraries. The two tools are unrelated and have different purposes. Given this difference in scope, we have retained the name PoolParty.

Reviewer 3

1. I think that the emphasis made on PoolParty being a universal tool that can “replace” (line 256) existing code/tools needs to be qualified more. The benefits of PoolParty are that it is straightforward to use, interpretable, modular, with a nice ‘design card’ tracking and graphed interpretation – I think this is particularly useful for ML and in silico work.

However, whilst it does streamline design processes, the package operates on input sequences rather than genomic loci co-ordinates/transcript so in terms of wet-lab experiments it is most useful for fundamental biology cDNA DMS and MPRA assays that are episomal (i.e. from plasmid / using reporters) rather than MAVES that rely upon genome editing (which is becoming the gold standard for clinical variant work). It does not intuitively and natively support reference genome coordinates, transcript models, exon/intron structures, alternative isoforms, or genome assembly awareness (as far as I can tell); VaLiAnT does do this. Base sequences into which variants are installed are represented/input as sequence string (pasted by user) rather than standardized co-ordinate or HGVS notation (this limits interpretation/ingress of variants from clinical and genomics resources). If PoolParty is presented as intuitive and adaptable rather than a universal replacement to alternative tools and code this is less of a concern and may allow researchers to more readily select which software to invest time into learning/using.

We agree that the original wording was too broad. We have removed language suggesting that PoolParty replaces other tools and revised the Background and Discussion to define its scope more clearly.

In Additional file 1, we have added two tables explicitly comparing PoolParty with existing tools (Tables S2 and S3). We have also added two figures showing side-by-side implementations of the same libraries in PoolParty and VaLiAnT or tangermeme (Figs. S3 and S4).

PoolParty does not currently accept HGVS notation. We discuss the broader genome- and transcript-awareness limitation under Comment 3 below. We have revised the relevant statements as follows:

Existing tools can help with parts of this process.

We have presented PoolParty, a Python package for designing oligonucleotide libraries. PoolParty facilitates this task by representing libraries as directed acyclic graphs (DAGs) that users can construct using a simple API.

PoolParty complements existing library-design tools such as VaLiAnT (Barbon et al., 2022) and MPRAinator (Georgakopoulos-Soares et al., 2017) by providing a single framework for DMS libraries, MPRA libraries, and library designs that combine elements of both.

Additional comparisons between PoolParty and existing tools are provided in Additional file 1: Figs. S3 and S4 and Tables S2 and S3.

2. Molecular consequence beyond codon change for missense is not reported as far as I can tell, VaLiAnT does this to some extent (e.g. stop-gained, synonymous, inframe deletion, etc, PoolParty may do this also, but I couldn't see this after running a 'replacement_scan'?) but generally researchers use VEP on their tool outputs to achieve extensive annotation. VEP requires standard annotation or format (e.g. VCF) for input rather than sequence strings – if this is possible for PoolParty (the production of a VCF or a format that VEP will accept) then one could obtain extensive annotations (e.g. SpliceAI predictions, EVE, AlphaMissense, PolyPhen, Consequence, genomic coordinates, Clinvar and gnomAD identifiers etc etc) which I think would greatly increase the uptake of the tool by researchers in the MAVE community that are more clinically/genomically minded – if this is possible I would state this/provide an easy means to generate VEP input format from PoolParty outputs.

Thank you for raising these points. PoolParty doesn't directly annotate molecular consequences for variants, but for ORF-aware Operations these can usually be inferred from the Operation used and the amino-acid changes recorded in the design cards (i.e., `wt_aas`, `mut_aas`). In some cases, the consequence is already specified by the user, e.g., when requesting missense-only, synonymous, or nonsense mutations using `mutagenize_orf`. We have also implemented ORF-aware insertion and deletion scans, with the insertion scan also supporting codon replacement, because the corresponding sequence-level Operations do not provide coding-aware design cards. These are local coding consequences only, though, since PoolParty does not track transcript context.

The suggestion to support VCF output is great. We can imagine two ways of implementing it. The first is to make PoolParty aware of genomic coordinates, such that every Operation can track coordinate changes and propagate them downstream. The difficulty is that many Operations can't do this cleanly because the coordinate changes are not well defined, e.g., `join`, `shuffle`, and `recombine`. The second is to implement an alignment algorithm that calls variants from the generated sequences post hoc, but this can be tricky for the same reason. It should be possible to support VCF output for a restricted set of Operations with well-defined coordinate changes, but we decided to leave this to a future version because it is not obvious what restrictions would be needed to prevent seemingly valid VCF output in unsupported cases. As an alternative, we implemented `from_vcf`, which extracts the corresponding sequence window and inserts the alt allele independently for each VCF record. The resulting sequences can then be used as input to genomic deep learning models or for clinically focused episomal MAVE assays.

3. The authors note limitations at the end of the discussion. I think updating this section to

address that PoolParty is mostly of benefit for DMS and MPRA rather than genome editing MAVE assays (e.g. not as useful for Saturation Genome Editing HDR libraries and Saturation Prime Editing pegRNA libraries) which require extensive and specific annotation/standardized nomenclature and consideration of transcript (generally through ingress of GTF/GFF files) and exon/intron boundaries (including split codons between exons, again through GTF/GFF boundary annotations).

Alternatively/additionally the authors might wish to highlight that the tool is agnostic to species/genome/transcript and present it as a more intuitive sequence generation tool which is particularly useful for ML testing/benchmarking which is well tracked with the design cards and interpretable/iteratively mutable through graph interrogation. This builds on the comment that generally annotations are variant specific (line 56-58), with the emphasis being that PoolParty serves as a useful tool to manifest oligo libraries as structured objects (less of a concern for wet-lab investigations in my opinion, but much more salient for ML based studies on raw sequence out of genomic context).

We agree and have updated the Limitations section accordingly. The revised text reads:

PoolParty operates on sequences, not genomic loci or transcript models. It can accept genomic coordinates as input to extract sequences, but does not interpret transcript annotations or keep track of those coordinates through subsequent design steps (e.g., as VaLiAnT does), which may make PoolParty less helpful for certain genome-editing applications.

4. Line 12 – there is not a “lack” of purpose-built software, replace with “scarcity” perhaps.

We have replaced “lack” with “scarcity.” The revised sentence reads:

Designing these libraries, however, remains tedious and error-prone due to the scarcity of purpose-built software.

5. Line 51-53 - stated that VaLinAnT is assay specific, then say used for DMS and saturation genome editing two different assays (SGE is a type of DMS, but VaLinAnT is used for SGE and cDNA DMS which are quite different assays).

We have replaced “assay-specific” with more general wording and revised the surrounding description. We also tested VaLiAnT on noncoding regions and confirmed that it supports these designs. We have revised the description as follows:

Several other tools automate library design or sequence perturbation for particular applications. For example, VaLiAnT (Barbon et al., 2022) generates mutational scanning libraries of both coding and noncoding regions; supported mutation types include single-nucleotide substitutions, amino acid substitutions, and in-frame deletions.

6. Line 66 – “expensive computation” this is only true for truly massive sequence generation – most full gene DMS/saturation libraries of >20,000 variants take <5 minutes on a standard laptop CPU (true of Valiant) – this statement would be true for many permutations of sequence for use in ML (e.g. pairwise, full genome CDS on GPU) – I would qualify this point on cost.

Indeed, most libraries designed for wet-lab experiments can be generated in seconds to minutes on a standard CPU, as the reviewer notes. We have therefore made the wording more specific:

Importantly, no sequences are generated until the user requests them, and it is simple to generate small test sets of sequences. This allows users to explore multiple design options without having to generate complete libraries at each iteration.

7. The use cases of protein coding, MPRA promoter and splice are good examples (line 68).

We are glad the reviewer found the three use cases well chosen; we selected them to illustrate the types of library designs where we think PoolParty is particularly useful. We appreciate the other positive feedback.

8. Figure 2 B and Figure 3 B – is it necessary helpful to put these code blocks here? I’m not sure how useful this is for readership.

We have retained these code panels because they show directly how the library designs in panels A are specified using PoolParty. The examples are short and also illustrate the relationship between the code and the corresponding DAGs and outputs.

9. Line 194 – ‘most abundant codon’ – this needs to be defined, is this codon usage (I think not) or instance in the input sequence?

`missense_only_first` selects the first codon listed for each target amino acid in the specified codon table, which by default is ordered by human codon-usage frequency. It does not count codons in the input sequence. We have clarified this in the text:

We note that `mutagenize_orf` supports multiple codon mutation types; this example uses `missense_only_first`, which selects the most frequently used codon in the human genome for each target amino acid.

10. Line 235 – SpliceAI is defined here for the first time but is introduced before this in section 2.8 – I would move the ‘a deep learning model of mRNA splicing’ to the earlier section.

We have defined SpliceAI in the earlier Implementation subsection where the model is first used. The revised text reads:

SpliceAI (Jaganathan et al., 2019) is a genomic deep learning model for splice-site prediction in humans. We used the five-model ensemble to predict the canonical 5’ss probability using ± 5 kb of genomic context (GRCh38).

11. Line 238 “varying strength” – maxentscan in legend, I would define this in the main body text too.

We have clarified in the Results that cryptic splice-site strength refers to the MaxEntScan score. The revised sentence reads:

Using the 5’ss of SMN2 exon 7 as the canonical site, we used PoolParty to construct a library in which cryptic 5’ss 9-mers of varying strength (MaxEntScan score) were

inserted at varying positions on either side of the canonical 5'ss (see Implementation).

12. Lien 248 – Fig.4G is introduced before 4E-F, perhaps generally introduce Fig.4. then this could stay as is.

We have moved the cross-validation result to the end of the paragraph so that the Figure 4 panels are introduced in order. The revised paragraph reads:

The GAMs closely reproduced the observed data (Fig. 4B,C). Cryptic sites on the exonic side suppressed canonical site usage more strongly than those on the intronic side (Fig. 4D), with the effect peaking approximately 20 bp upstream of the canonical 5'ss. This is consistent with the observation that cryptic 5'ss are preferentially depleted near annotated ones on the exonic side (Dawes et al., 2022). Stronger cryptic sites drove greater suppression in both models (Fig. 4E). The effects of position and strength exhibited substantial interactions (Fig. 4F). Through a comparison to analogous linear models, we observed that the non-linear terms and interaction terms in the GAMs substantially improved predictive performance, with held-out R^2 of 0.82 for exonic and 0.71 for intronic insertions (Fig. 4G).

13. Line 256 – “replaces the ad hoc” is too bold and a little presumptive as for major points above—needs clarification/qualification. Also, Line 274 – “in contrast”— I think these claims are too strong – valiant can do DMS and MPRA as well as SGE/SPE – I would tone these statements down and/or emphasise what is unique to PoolParty.

As discussed in our response to Comment 1 above, we have revised both statements:

We have presented PoolParty, a Python package for designing oligonucleotide libraries. PoolParty facilitates this task by representing libraries as directed acyclic graphs (DAGs) that users can construct using a simple API.

PoolParty complements existing library-design tools such as VaLiAnT (Barbon et al., 2022) and MPRAinator (Georgakopoulos-Soares et al., 2017) by providing a single framework for DMS libraries, MPRA libraries, and library designs that combine elements of both.

14. Line 271 – “in the spliceAI example” – the one in section 2.8 or 3.3? (3.3 I imagine, but best to state).

We have clarified that this refers to the SpliceAI surrogate modeling example in the Results. The revised sentence reads:

For example, in the SpliceAI analysis of Fig. 4, the cryptic splice-site strength and position recorded in each design card provided the covariates used for surrogate modeling.

15. The code is very well written and documented, and it is noted by the authors that Anthropic models have been used to do this/help which I think is sensible, editorially this may be something

to confirm is acceptable at Bioinformatics (as reviewers were requested not to use these tools, but this is likely due to confidentiality).

We checked the current BMC policy and confirmed that this use is permitted when disclosed. The software was developed through human coding and testing with assistance from LLMs. LLMs were also used for copy-editing the manuscript, while the scientific content and conclusions were determined and reviewed by the authors.

16. The name ‘PoolParty’ is great.

Thank you!

References

- Swainston N, Currin A, Green L, Breitling R, Day PJ, Kell DB. CodonGenie: optimised ambiguous codon design tools. *PeerJ Comput Sci.* 2017;3:e120. <https://doi.org/10.7717/peerj-cs.120>.
- Zulkower V, Rosser S. DNA Chisel, a versatile sequence optimizer. *Bioinformatics.* 2020;36(16):4508–4509. <https://doi.org/10.1093/bioinformatics/btaa558>.
- Schreiber J, Lorbeer FK, Heinzl M, Lu YY, Stark A, Noble WS. Programmatic design and editing of cis-regulatory elements. *bioRxiv.* 2025;p. 2025.04.22.650035. <https://doi.org/10.1101/2025.04.22.650035>.
- Barbon L, Offord V, Radford EJ, Butler AP, Gerety SS, Adams DJ, et al. Variant Library Annotation Tool (VaLiAnT): an oligonucleotide library design and annotation tool for saturation genome editing and other deep mutational scanning experiments. *Bioinformatics.* 2022;38(4):892–899. <https://doi.org/10.1093/bioinformatics/btab776>.
- Georgakopoulos-Soares I, Jain N, Gray JM, Hemberg M. MPRAinator: a web-based tool for the design of massively parallel reporter assay experiments. *Bioinformatics.* 2017;33(1):137–138. <https://doi.org/10.1093/bioinformatics/btw584>.
- Jaganathan K, Panagiotopoulou SK, McRae JF, Darbandi SF, Knowles D, Li YI, et al. Predicting Splicing from Primary Sequence with Deep Learning. *Cell.* 2019 Jan;176(3):535–548.e24. <https://doi.org/10.1016/j.cell.2018.12.015>.
- Dawes R, Joshi H, Cooper ST. Empirical Prediction of Variant-Activated Cryptic Splice Donors Using Population-Based RNA-Seq Data. *Nat Commun.* 2022 Mar;13(1):1655. <https://doi.org/10.1038/s41467-022-29271-y>.